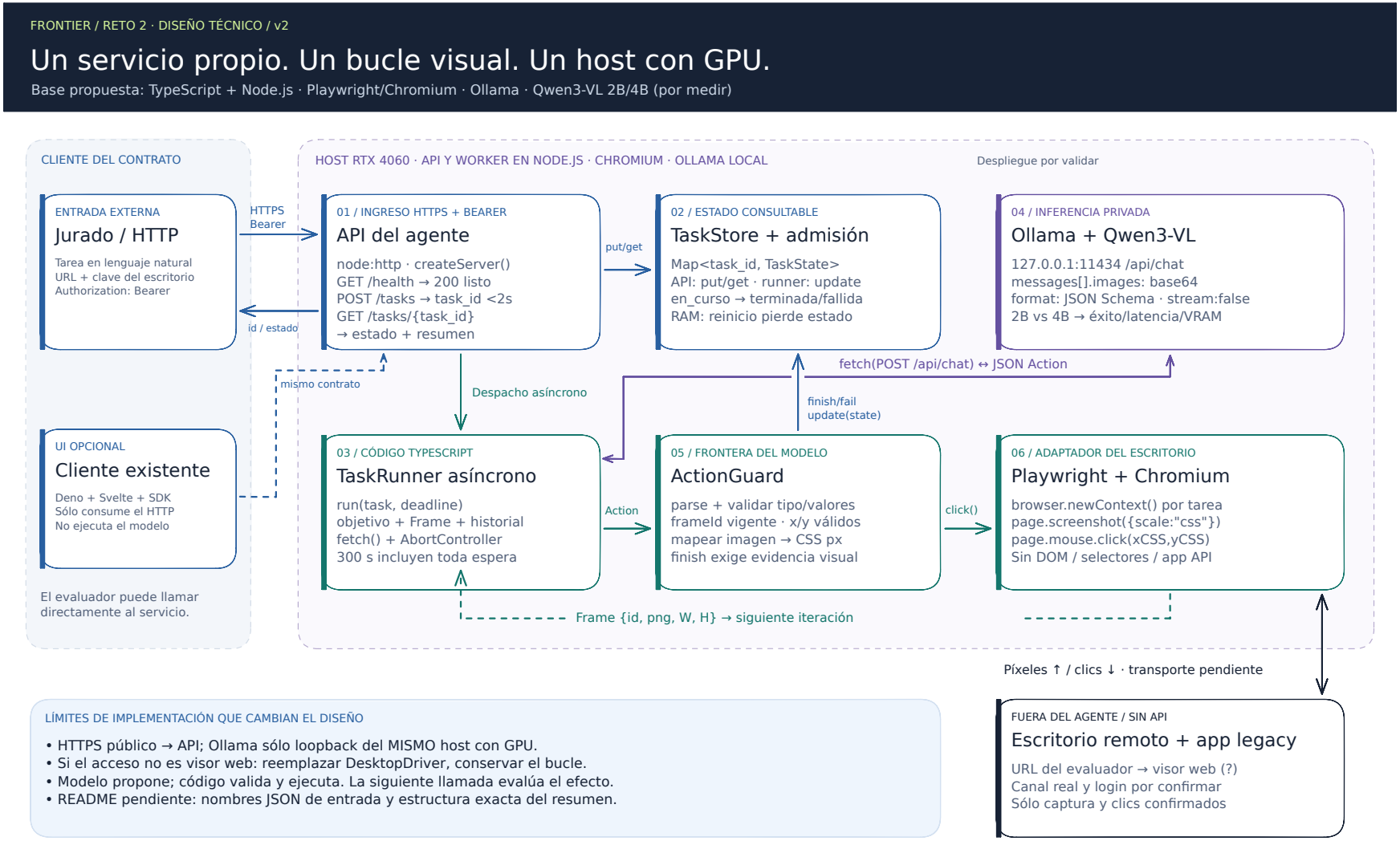
Arquitectura ejecutable

Mapa del contrato público, los módulos del servicio, la inferencia local y el canal permitido hacia la aplicación. Las cajas representan responsabilidades técnicas.



De píxeles a una acción validable

El modelo interpreta y propone. El servicio conserva sesión, estado y plazo; el controlador realiza la interacción. El mismo modelo evalúa el efecto en la siguiente iteración.

A / Captura → inferencia local

```
POST http://127.0.0.1:11434/api/chat
{
  "model": "qwen3-vl:2b",
  "messages": [{
    "role": "user",
    "content": "Objetivo + tamaño + historial breve",
    "images": ["<PNG codificado en base64>"]
  }],
  "stream": false,
  "format": "<JSON Schema de Action>"
}
```

Esquema ilustrativo: `format` recibe el objeto JSON Schema real, no ese texto. El tag 2B es candidato; la prueba puede seleccionar 4B.

Frame: datos del controlador

`id`, PNG, ancho y alto de entrada; geometría de la captura y momento de observación. El adaptador asocia la respuesta a ese Frame. `message.content` se analiza y valida antes de usar sus valores.

B / Inferencia → propuesta interna

```
type Action =
  | { kind: "click"; x: number; y: number }
  | { kind: "finish"; evidence: string }
  | { kind: "fail"; reason: string };

// Action es nuestro contrato interno propuesto.
// No reemplaza el JSON público del README.
```

C / ActionGuard antes de DesktopDriver

1. Validar tipo, números finitos y valores requeridos.
2. Asociar la propuesta al último Frame y al objetivo.
3. Comprobar coordenadas, plazo y estado `en_curso`.
4. Ejecutar un clic; obtener una observación nueva.
5. Aceptar `finish` sólo con evidencia pertinente; no basta con haber hecho clic.

Un `frameId` no garantiza que la pantalla no haya cambiado mientras responde el modelo. Popups, cambios de foco y animaciones requieren reobservación; no existe transacción visual atómica.

GEOMETRÍA / Las coordenadas terminan en píxeles CSS

1280 × 720
Viewport fijo; screenshot({scale: css})

→

x = 800, y = 300
Baseline: entrada completa = CSS; identidad

→

```
await page.mouse.click(800, 300);
// Después: screenshot() → modelo otra vez.
```

Si se añade recorte o reducción: xCSS = leftCSS + xModelo × anchoRecorteCSS / anchoEntrada; análogo para y. El prompt define unidades de entrada; no se asume la convención de coordenadas del modelo. No se añade redimensionado al baseline.

Tecnología, mecanismo y trade-off

Recomendación técnica de esta revisión: backend TypeScript sobre Node.js por el camino documentado de Playwright. El runtime Deno/Svelte del cliente permanece intacto. Versiones exactas se fijan tras el smoke test.

Responsabilidad	Tecnología propuesta	Mecanismo concreto	Razón y límite
API HTTP	Node.js + `node:http`	`createServer()` enruta health/tasks; autentica Bearer, acota el body y devuelve el id sin esperar al VLM.	Sin framework HTTP adicional. FastAPI/ Python añade otro runtime sin una ventaja demostrada para este cliente.
Estado y ejecución	`Map` + TaskRunner async	API hace put/get; runner actualiza estados. I/O asíncrona para captura e inferencia; un ejecutor inicial.	Sin Redis ni cola distribuida. Un proceso: reiniciar pierde estado; concurrencia/ admisión deben ajustarse al README.
Pantalla y clic	Playwright + Chromium	`newContext()` aísla tareas; `screenshot({scale:"css"})` captura; `mouse.click()` actúa por coordenadas.	Adecuado si el escritorio llega por visor web. No sirve por sí solo para un protocolo nativo distinto: cambiar sólo DesktopDriver.
Inferencia	Ollama + `fetch()`	`/api/chat`: imágenes base64, JSON Schema, `stream:false`. La respuesta entra en ActionGuard.	Modelo local evita tarifa por llamada; exige memoria y latencia suficientes. No necesita SDK de agentes, ni segundo planificador/ verificador.
Validación	Type guards TypeScript	Comprobar unión Action, números finitos, límites, frame y estado antes de efectos.	Los tipos de compilación no validan JSON externo. Sin librería extra al inicio; no interpretar la confianza declarada como certeza.
UI existente	Deno + Svelte + SDK	El cliente HTTP presenta tarea/estado; el jurado también llama directo al servicio.	No cambiar configuración, scripts ni arquitectura de UI. La interfaz no es dependencia de ejecución del agente.

CAMINO CRÍTICO EN NIXOS

Probar binario Chromium y dependencias del entorno; fijar un ejecutable compatible por configuración. Abrir el visor real, capturar y verificar un clic antes de programar el resto. El nombre del sistema legacy no revela su transporte.

QUÉ NO ENTRA EN ESTA IMPLEMENTACIÓN

OCR separado, framework de agentes, macros por aplicación, acceso DOM y control comercial completo. Rust/Burn sólo vuelve a evaluarse con un cuello de botella medido y compatibilidad del modelo demostrada; no es un requisito del MVP.

El código mantiene las garantías

Un único host con la RTX 4060 simplifica la comunicación entre API, navegador y Ollama. La disponibilidad pública y el transporte del escritorio siguen siendo puertas de validación.

INGRESO PÚBLICO

HTTPS + Bearer → API del servicio. `GET /health`: 200 al estar listo. `POST /tasks`: id en menos de 2 s, antes de ejecutar. `GET /tasks/{task_id}`: estado y resumen.

HOST RTX 4060

Node.js + Chromium + Ollama en la misma máquina. Sólo el ingreso HTTP se publica. Ollama usa `127.0.0.1:11434`; el puerto de inferencia no se expone.

RECURSOS Y OPERACIÓN

Mantener modelo cargado y comprobar readiness. Un contexto por tarea. Cerrar la sesión al terminar; credenciales fuera de logs y resúmenes. La app remota conserva sus propios datos.

PLAZO / Desde recepción, incluso si espera recursos

```
const deadline = performance.now() + 300_000; // tiempo monotónico
// Cada captura, fetch, espera y clic consulta el tiempo restante.
// Watchdog: publicar fallida al vencer; impedir acciones o cierres tardíos.
// AbortController corta la espera HTTP; no garantiza detener el cálculo de Ollama.
```

Estado público y resultado

en_curso
Tarea aceptada; observación/ decisión/acción

→

TERMINAL

`terminada` + evidencia del objetivo
`fallida` + motivo concreto

Un terminal no vuelve a `en_curso`. El runner actualiza TaskStore; el polling de HTTP lee ese mismo estado. Una propuesta tardía del modelo se descarta. La evidencia visual es una comprobación heurística; no equivale a una consulta de base de datos.

Sin cola oculta de varios minutos: política de ocupado/rechazo/concurrencia sujeta al README. Con un ejecutor no se promete atender tareas simultáneas.

Recuperación que sí modifica el recorrido

Evento	Respuesta del código
JSON inválido	Reintento acotado dentro del plazo; no ejecutar datos sin validar.
Sin cambio / bucle	Reobservar y replantear; detectar repetición y detener.
Escritura incierta	Leer evidencia visual antes de repetir para evitar duplicar.
Pantalla ambigua	No adivinar entidad ni éxito; nueva observación o fallo explicable.
Timeout / sesión rota	Estado terminal, cierre del contexto y bloqueo de acciones tardías.

Elegir por evidencia antes de crecer

Este es un plan de medición: no se han ejecutado aún estas pruebas en el escritorio real ni el benchmark de los modelos. Los umbrales internos no son la rúbrica oficial del jurado.

Puerta	Prueba concreta	Métrica / evidencia	Decisión
G0 / Transporte	Conectar, capturar y accionar un control conocido; verificar el cambio. Repetir en sesión limpia.	Captura correcta, coordenadas CSS correctas y efecto visible.	Si falla, corregir DesktopDriver antes de construir el agente.
G1 / Grounding	Mismas 10 pantallas para 2B/4B: botones, factura, cliente y valores. Referencia manual con zonas válidas.	Aciertos/10, errores de clic y lectura; tiempo frío/caliente por llamada.	Meta inicial ≥8/10 localizaciones por candidato; no es prueba de generalización.
G2 / Tarea completa	4 tareas × 2 repeticiones × 2 modelos: abrir factura correcta, desambiguar, navegar diálogo y verificar cierre.	Éxito por caso, llamadas, clics erróneos, falsos cierres, duración y pico VRAM.	Meta interna ≥80% éxito y 0 falsos cierres. Elegir sólo un candidato que cumpla el plazo.
G3 / Contrato	Desde internet: Bearer válido/inválido, health, crear y consultar; forzar timeout y respuesta tardía.	Creación menos de 2 s; terminal ≤300 s; sin acciones tras el cierre. Registrar todas las llamadas.	Contrato y límites en todas las tareas aceptadas; revisar códigos/JSON contra README.
G4 / Robustez	App nueva, sesión limpia, nombres parecidos, fecha de la app distinta a hoy, filas fuera de vista y popup inesperado.	Errores por escenario y evidencia del resultado; no basta contar clics.	No usar rutas fijas ni asumir que lo visible cubre el total.

COSTE Y LATENCIA

Medir captura, inferencia, acción y espera por separado. Registrar p50/p95 con tamaño de muestra, arranque frío, tokens/llamadas y memoria. El coste de inferencia local es tiempo/energía/hardware; no hay tarifa por llamada a un proveedor.

Elegir el modelo que resuelva más tareas dentro del presupuesto; si ninguno pasa, revisar viabilidad antes de añadir componentes.

PENDIENTES QUE DESBLOQUEAN IMPLEMENTACIÓN

README: campos exactos de entrada, resumen y evaluación. Transporte/login del escritorio y permisos de teclado/scroll. Concurrencia y admisión. Instalación de Ollama, memoria real disponible y publicación HTTPS del host. Sólo captura y clics están confirmados.

Orden: documentar decisiones → validar G0/G1 → implementar ciclo mínimo → medir G2/G3 → ampliar por fallos observados.

Referencias oficiales consultadas: Node.js HTTP · Playwright Library · Mouse / píxeles CSS · Ollama chat / JSON Schema · Ollama vision / base64 · Qwen3-VL. Brief, límites y rúbrica: material proporcionado por el participante.